Bachelorstudiengang Software Design & Cloud Computing SWD24W (BA, BB)

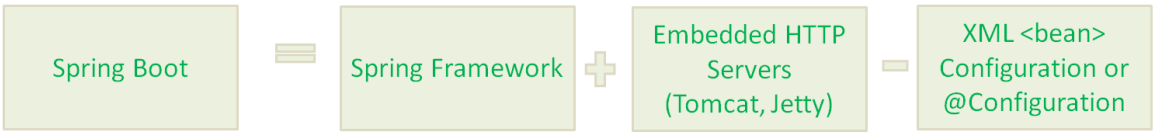
„Software Architekturen“ | DI Dr. Gaube Gerhard | Sommer 2026

Lab1 | Microservice Setup, REST and DB configuration

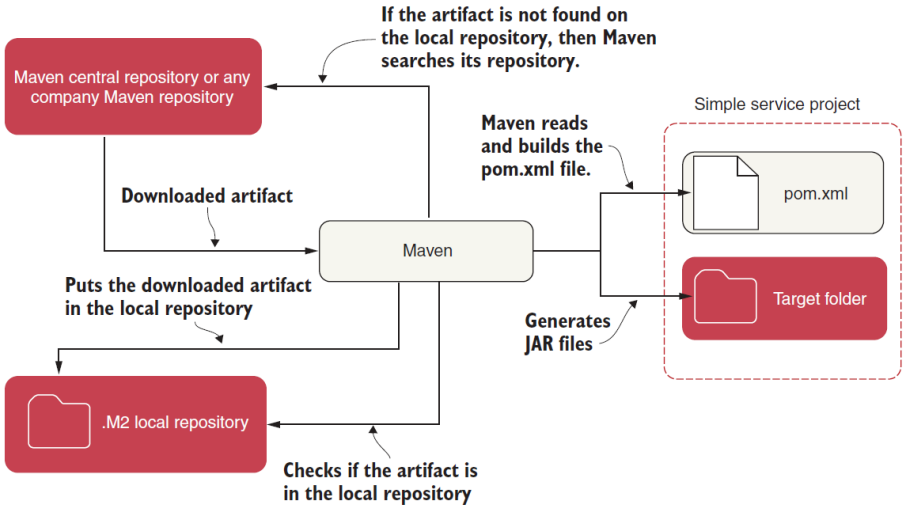
In this Lab, we will setup and configure a basic microservice, add REST controller and various databases.

Technology Stack Spring Boot

- Java 21 with Project Management Tool Maven
- Spring Boot and Cloud
- HTTP Client (i.e. Postman) and Open API with Swagger



Software Projectmanagement Tool Maven



Project Source Code:

Code for all Labs is available at: https://github.com/ggaube/swd24-software-architecture

Task 1 Microservice Setup und Configuration

Setup, Configuration and Test of a Microservice

1. Setup and configuration with *Spring Initializr* https://start.spring.io/
2. Add project metadata, such as project coordinates group ID, artifact ID, and version, language packaging type, and Java version
3. Add at *Selection of dependencies* the dependency `Spring Web`
4. Generate project and import it into Java IntelliJ or other IDE
5. Edit the configuration file `application.properties`
    a) Change the server port to 4000
    b) Give the service a name
6. Start the service
7. Add the Java class `TestController.java`
8. Send a request to the endpoint with a HTTP client (Postman)

Open API Controller Documentation

1. Check the URL: https://springdoc.org/
2. Add the dependency of section "2 Getting Started" to the POM file
3. Restart the service and go to: http://localhost:5000/swagger-ui/index.html

Coding Tasks and Control Questions

1. Browse through various available dependencies
2. Which static method starts the container?
3. What annotation is used to generate the container and what is its meaning?
4. What special term does Spring Boot use for container?

Task 2: In-Memory DB H2 in a Spring Boot Application

Most applications need to interact with a database to store and retrieve application data. However, before communicating with the database, you need to configure and access a relational database in your Spring Boot application.

Spring Data with Jakarta Persistence API (JPA) makes it easy to implement code to access any repository at the data access layers. *Repository interfaces* can be designed including custom CRUD methods. This enables the implementation for an in-memory-database which can easily be replaced by a persistent database later.

SOLUTION

To configure a relational database with Spring Boot, you can add `spring-bootstarter-data-jpa` and the relational database driver dependency in the `pom.xml` of your application. Additionally, you need to provide the database details, such as database username, password, driver class, and connection URL.

Service Dependencies

- Spring Web `spring-boot-starter-web`
- Spring Data JPA `spring-boot-starter-data-jpa`
- Apache Derby Database `H2 database`

Coding Tasks I

1. Check the `application.proerties` configuration file to connect to the H2 database with the correct credentials
2. Start the project and connect via the H2 Console to the database: http://localhost:5000/h2-console
3. Check the data and send arbitrary SQL statements to the database

```
INSERT INTO CUSTOMER (id, first_name, last_name)
VALUES (1, 'John', 'Doe');
```

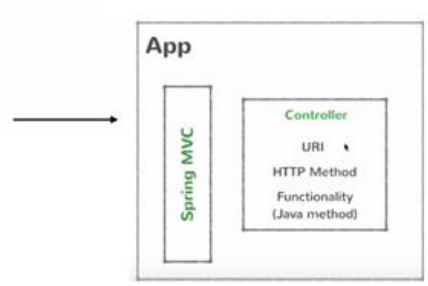
Task 3: REST API with JPA

We will implement a Spring Boot microservice with REST endpoints and a JPA *in-memory derby* database configured and build with Maven. In the next step, the H2 database will be replaced by a persistent SQL database.

Guide and Tutorial: https://spring.io/guides/gs/accessing-mongodb-data-rest

REST has become the de-facto standard for building web services because it is easy to build and easy to consume for any type of applications. REST is not a standard per se, but an approach, a style, a set of constraints on the architecture that can help to build highly distributed web-scale systems. By building on top of HTTP, REST APIs provide these advantages:

- Standard GET, POST, PUT, DELETE methods
- Backwards compatible and evolvable APIs
- Scalable and securable services



Patient REST API with database

Setup and configure a Java Maven project with these dependencies and Java classes:

1. **Service Dependencies**
   - Spring Web `spring-boot-starter-web`
   - Spring Data JPA `spring-boot-starter-data-jpa`
   - Apache Derby Database `H2 database`
   - `OpenAPI (springdoc.org)` `springdoc-openapi-starter-webmvc-ui`

2. **Java Classes**
   - Patient.java — Data Modell
   - PatientService.java — Java business functionality
   - PatientController.java — REST endpoints
   - PatientRepository.java — Interface for JPA
   - PatientApp.java — Main function to start service

3. **Global service configuration in** application.properties
   a. Every service should have a name
      `spring.application.name = patient-client`

   b. The Service runs under a defined (not well-known) port:
      `server.port= 5000`

   c. Add and configure H2 console for the In-memory database

4. **GET, POST, PUT and DELETE Request**

   a. Check the endpoints in the Swagger UI: http://localhost:5000/swagger-ui/index.html
   b. At the beginning, we do not have any data in our system. Hence, we must send a JSON POST request to the POST-endpoint. Check the result of the successful upload with a GET request:

```
{
    "patient": "Donald",
    "diagnosis": "no help possible"
}
```

   c. To manipulate the available data, test the PUT and DELETE endpoints. Check the necessary parameters to enable the correct requests to the endpoints.

5. **Configuration of mySQL DB and Hibernate**

After configuration and test of H2 in-memory embedded database, a persistent MySQL Database with Hibernate will be configured in this section. The interface of the existing Spring Data JPA will be used to access the new database:

- XAMPP with phpMyAdmin for administration – in this Lab we will use WAMPP: https://www.apachefriends.org/de/download.html
- Docker container
- Any other SQL databse with administration console

Spring Hibernate provides these configurations. See the documentation for details:

- **none**: Default for MySQL - no change is made to the database structure.
- **update**: Hibernate changes the database according to the given entity structures.
- **create**: Creates the database every time but does not drop it on close.
- **create-drop**: Creates the database and drops it when SessionFactory closes.

You must begin with either *create* or *update*, because you do not yet have the database structure. After the first run, you can switch to *update* or *none*, according to program requirements. Use *update* when you want to make some change to the database structure.

a. Service Dependencies for DB

To enable a connection to the SQL database we must provide a mySQL connector:

- MySQL Driver `mysql-connector-java`

b. Applikation Properties for DB

This configuration routes the data to the database and is configured in the `application.properties` file. The database `patient` must be created in the mySQL database, the mapping of the Java data model is automatically done by Hibernate.

```
spring.jpa.hibernate.ddl-auto=update
spring.datasource.username=root
spring.datasource.password=

# MySQL Port 3306
#spring.datasource.url=jdbc:mysql://${MYSQL_HOST:localhost}:3306/patient

# MariaDB Port 3308
#spring.datasource.url=jdbc:mysql://${MYSQL_HOST:localhost}:3308/patient
```

c. Patient Service DB Test

a. Generate data with POST requests and check the result in the mySQL database.

b. Delete some data in the database and check the result with a GET request to the service.

c. Compare the data model in the Java code with the structure in the database.

d. Coding Tasks and FAQs

a. Extend the data model in the patient-service by some parameters and adapt the code.

b. Check all annotations for database functionality (table, attributes, key)

c. In which class can we see the connection between JPA and CRUD implementation? Check the available library methods.

d. How is the error-handling of HTTP response-request operations done?

e. Check and describe the concept of Hibernate

Task 4 Atlas MongoDB for REST Spring Boot Application

Along with relational databases, NoSQL databases are gaining popularity. MongoDB is a popular NoSQL database that stores the data as documents in JSON-like format. Spring Boot provides with MongoDB Compass a local and with MongoDB Atlas a cloud based approach to use a noSQL database.

Configuration Tasks for MongoDB Atlas

1. Configure a sign in at: https://www.mongodb.com/products/platform/atlas-database
2. Configure Network and Database access
3. Configure Cluster and Connection
4. Test the connection from your client to your cluster

Coding Tasks

1. Ckeck the configuration in the `application.properties` file and modify your connection string
    a. Cloud based MongoDB Atlas
    b. Local Compass MongoDB
2. Check all Java classes (Lombok Plugin is needed)
3. Setup and test the service with an arbitrary connection string
4. Check the Swagger UI, test some requests and check the results in the database
5. Use a connection string to access the database of your colleges and manipulate data